BACKEND DEVELOPMENT

Phase 1: Complete JavaScript Prerequisites

Seven lessons | Study notes, executable examples, pitfalls, exercises, solutions and quick reference

Purpose. A consolidated learning reference for progressing from basic JavaScript to Node.js, Express, PostgreSQL, NestJS, GraphQL, Socket.IO, LangChain and LangGraph. All code assumes a recent Node.js LTS release unless stated otherwise.

How to use. Read chapters 1-7 in sequence, type and run the examples, predict outputs before checking, and complete the final TaskFlow API exercise. Node.js examples using `import` assume `"type": "module"` in package.json.

Contents

- 1. Modules: CommonJS, ES Modules and exports
- 2. Callbacks and error-first APIs
- 3. Promises, chaining, microtasks and Promise.all
- 4. Async/await, try/catch and concurrent I/O
- 5. Event loop, timers, I/O and CPU blocking
- 6. Error handling and Express middleware
- 7. Modern syntax, configuration and a modular Express API
- Appendix A. Predict-the-output questions and answers
- Appendix B. Common bugs and backend quick reference
- Appendix C. TaskFlow exercises and solution outline

Note on examples: Simulated database operations use timers for teaching. They are not actual database calls. For real user data, validate inputs, enforce database constraints and keep secrets out of source control.

1 | JavaScript Modules

1.1 Why modules exist

A module separates a backend into smaller files with focused responsibilities: routing, controller functions, database access and configuration. A module exports selected values; another module imports them. This reduces coupling and makes code easier to test.

1.2 CommonJS: require and module.exports

```
// math.cjs
function add(a, b) { return a + b; }
function multiply(a, b) { return a * b; }
module.exports = { add, multiply };

// main.cjs
const { add, multiply } = require('./math.cjs');
console.log(add(5, 3));       // 8
console.log(multiply(5, 3));  // 15
```

Key distinction: `require("./math.cjs")` loads the exported object. Curly braces on the left perform object destructuring.

1.3 ES Modules: import/export

```
// package.json (relevant part)
{ "type": "module" }

// math.js
export function add(a, b) { return a + b; }
export function multiply(a, b) { return a * b; }

// main.js
import { add, multiply } from './math.js';
console.log(add(5, 3));       // 8
```

For relative ES Module imports in Node.js, include the filename extension. A named export is imported with braces; a default export is imported without braces.

```
// user.js
export const role = 'admin';
export default function getUser() { return { id: 1 }; }

// main.js
import getUser, { role } from './user.js';
console.log(getUser(), role);
```

Feature	CommonJS	ES Modules
Import	require()	import ... from ...
Export	module.exports	export / export default
Configuration	Usually .cjs or default CommonJS	"type": "module" or .mjs
Typical use	Existing Node.js projects	Modern JavaScript projects

1.4 Backend example

```
// tasks.js
export function getTask() {
  return { id: 1, title: 'Learn Node.js', completed: false };
}

// server.js
import http from 'node:http';
import { getTask } from './tasks.js';
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(getTask()));
});
server.listen(3000);
```

Exercise: Export a function `getUser()` from `user.js` and import it into `server.js`. The expected result is an object containing a name.

2 | Callbacks

2.1 Definition and basic execution

A callback is a function passed to another function, which invokes it when appropriate. Passing a function is different from calling the function.

```
function greet(name) { console.log('Hello ' + name); }
function processUser(callback) {
  const username = 'Rahim';
  callback(username);
}
processUser(greet); // Hello Rahim
// processUser(greet()) calls greet immediately and
// passes its return value instead of a callback.
```

2.2 Synchronous vs asynchronous

```
console.log('1. Start');
[10, 20].forEach(n => console.log(n));
console.log('2. End');
// 1. Start, 10, 20, 2. End (synchronous)

console.log('A');
setTimeout(() => console.log('B'), 1000);
console.log('C');
// A, C, then B after approximately 1 second
```

A callback is not automatically asynchronous. `forEach` calls its callback synchronously; `setTimeout` schedules a callback for later.

2.3 Node.js error-first callbacks

```
import fs from 'node:fs';
console.log('1. Start');
fs.readFile('./users.json', 'utf8', (err, data) => {
  if (err) {
    console.error('File error:', err.message);
    return;
  }
  console.log('3. Parsed user:', JSON.parse(data));
});
console.log('2. Other work');
```

The callback receives `err` first (usually null on success) and data second. Node.js can continue running code while the file read is pending. If file contents may be invalid JSON, also handle parsing errors.

2.4 HTTP request handlers are callbacks

```
import http from 'node:http';
http.createServer((req, res) => {
  res.end('Hello');
}).listen(3000);
// (req, res) => { ... } executes for each request.
```

2.5 Callback hell

```
getUser(1, user => {
  getOrders(user.id, orders => {
    getPayment(orders[0].id, payment => {
      console.log(payment);
    });
  });
});
```

Dependent callbacks create growing indentation, repeated error paths and difficult control flow. Promises and async/await let you express the same sequence more clearly.

Exercise: Write `processTask(name, callback)` that logs `Processing: Learn Node.js` and invokes the callback to log `Task completed!`.

3 | Promises

3.1 The three states

A Promise represents an eventual result. It begins pending and settles as either fulfilled (value) or rejected (reason). Once settled, its state is fixed.

```
const p = new Promise((resolve, reject) => {
  const success = true;
  if (success) resolve('Data loaded');
  else reject(new Error('Load failed'));
});
p.then(value => console.log(value))
 .catch(error => console.error(error.message));
```

The Promise executor runs immediately. However, handlers supplied to `.then()` and `.catch()` run asynchronously as Promise microtasks, even for an already-settled Promise.

3.2 Understanding order

```
console.log('1');
Promise.resolve('ready').then(v => console.log('3', v));
console.log('2');
// 1, 2, 3 ready
```

3.3 Simulated database call

```
function getUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id === 1) resolve({ id: 1, name: 'Rahim' });
      else reject(new Error('User not found'));
    }, 1000);
  });
}
getUser(1)
  .then(user => console.log(user.name))
  .catch(error => console.log(error.message));
```

3.4 Chaining dependent operations

```
getUser(1)
  .then(user => getOrders(user.id))
  .then(orders => getPayment(orders[0].id))
  .then(payment => console.log(payment))
  .catch(error => console.error(error));
```

Returning the nested Promise is important: the next `.then()` adopts its outcome and waits for its fulfillment. Forgetting `return` may cause the next handler to run with `undefined`.

3.5 Promise.all for independent work

```
const profile = Promise.resolve({ name: 'Rahim' });
const settings = Promise.resolve({ theme: 'dark' });
Promise.all([profile, settings])
  .then(([user, config]) => console.log(user, config))
  .catch(console.error);
```

`Promise.all()` preserves result ordering, waits for all inputs to fulfill, and rejects if any rejects. It coordinates concurrency; it does not itself create worker threads. Use when tasks are independent and all results are required.

3.6 Other coordination methods (reference)

Method	Behavior
Promise.all	All must fulfill; rejects on first rejection.
Promise.allSettled	Waits for all; reports each fulfillment/rejection.
Promise.race	Settles as soon as one input settles.
Promise.any	Fulfills with first fulfillment; rejects if all reject.

Exercise: Create `checkStock(productId)`, wait 1 second, fulfill for ID 10 and reject for other IDs. Handle both outcomes.

4 | Async / Await

4.1 async always returns a Promise

```
async function getNumber() { return 10; }
console.log(getNumber()); // Promise { 10 }
getNumber().then(n => console.log(n)); // 10
```

`await` obtains a fulfilled value or throws the rejection reason. It suspends its current async function; it does not block the whole Node.js process.

4.2 Predicting output

```
function getData() {
  return new Promise(resolve => {
    setTimeout(() => resolve('Database result'), 2000);
  });
}
async function main() {
  console.log('1. Query starts');
  const data = await getData();
  console.log('2.', data);
}
main();
console.log('3. Other code');
// 1. Query starts, 3. Other code, 2. Database result
```

4.3 try/catch and finally

```
async function showUser() {
  try {
    const user = await getUser(999);
    console.log(user);
  } catch (error) {
    console.error('Error:', error.message);
  } finally {
    console.log('Operation finished');
  }
}
showUser();
```

If an awaited Promise rejects, execution jumps to the nearest applicable catch block. Without appropriate handling, an async function itself returns a rejected Promise.

4.4 Sequential vs concurrent

```
// Sequential: second begins after first is fulfilled.
const user = await getUser();
const products = await getProducts();

// Concurrent: both operations are started immediately.
const [user2, products2] = await Promise.all([
  getUser(),
  getProducts()
]);
```

If the independent operations take ~2 and ~3 seconds, sequential elapsed time is roughly 5 seconds; concurrent elapsed time can be roughly 3 seconds, excluding overhead. Dependent operations must remain ordered.

4.5 Async Node HTTP handler

```
import http from 'node:http';
const server = http.createServer(async (req, res) => {
  try {
    const user = await getUser(1);
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(user));
  } catch (error) {
    res.writeHead(500);
    res.end('Internal server error');
  }
});
server.listen(3000);
```

Production code should differentiate missing records (usually 404) from unexpected server failures (500). This short example is only to demonstrate control flow.

4.6 Mistakes to avoid

- Forgetting await: the variable holds a Promise rather than the data.
- Awaiting independent operations one at a time unnecessarily increases total latency.
- Calling an async function without observing its returned Promise can create an unhandled rejection.
- Adding async to a CPU-heavy function does not make its synchronous computation non-blocking.

Exercise: Implement `getOrderDetails(101)`, resolving to `{id:101,price:500}` after 1 second and rejecting otherwise. Consume it in `async main()` with try/catch.

5 | The Node.js Event Loop

5.1 Stack, runtime and scheduling

The call stack runs JavaScript function calls. Node.js uses V8 to execute JavaScript and libuv/operating-system facilities to manage asynchronous I/O. The event loop coordinates when callbacks are executed. An asynchronous request may progress while the main JavaScript thread handles other work.

```
function c() { console.log('C'); }
function b() { c(); }
function a() { b(); }
a();
console.log('Done'); // C, Done
```

Function calls push stack frames; returns pop frames. A long synchronous operation keeps the main JS thread occupied and delays other callbacks.

5.2 Promise microtasks and timers

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
// A, D, C, B
```

Synchronous code runs first. Promise reaction callbacks run as microtasks after the current JavaScript completes. The zero-delay timer executes later when eligible. A 0 ms timeout does not run immediately.

5.3 process.nextTick

```
// Save as example.cjs (CommonJS context)
console.log('A');
process.nextTick(() => console.log('B'));
Promise.resolve().then(() => console.log('C'));
console.log('D');
// A, D, B, C (in this CommonJS example)
```

Do not generalize that nextTick always precedes every Promise callback: top-level behavior can differ in ES Module evaluation. Recursive microtasks or nextTick callbacks can starve I/O.

5.4 Important event-loop phases

Phase	Role
Timers	Eligible setTimeout and setInterval callbacks
Pending callbacks	Certain deferred system callbacks
Poll	I/O activity and related callbacks
Check	setImmediate callbacks
Close callbacks	Some socket/handle close events

At top level, the ordering of zero-delay `setTimeout` and `setImmediate` is not guaranteed. Inside a completed I/O callback, setImmediate normally executes before a newly scheduled zero-delay timer. Timer scheduling behavior changed in Node.js 20/libuv 1.45.

```
// io.cjs
const fs = require('node:fs');
fs.readFile(__filename, () => {
  setTimeout(() => console.log('timer'), 0);
  setImmediate(() => console.log('immediate'));
});
// Typically: immediate, timer
```

5.5 Concurrency vs parallelism

Concurrency: operations make progress during overlapping periods. Parallelism: work executes simultaneously on multiple processing resources. Promise.all can overlap timers and I/O without executing two main-thread JavaScript callbacks simultaneously.

5.6 CPU blocking demonstration

```
function blockCPU(ms) {
  const end = Date.now() + ms;
  while (Date.now() < end) {}  // blocks main JS thread
}
blockCPU(3000);
console.log('Finished');
// An async keyword would NOT make blockCPU non-blocking.
```

Use asynchronous APIs for I/O, and consider worker_threads, dedicated workers or separate processes for CPU-intensive computation. A Node HTTP handler waiting on a database query is not using the main JS thread to execute the query.

Exercise: Predict the output of synchronous logs, a Promise handler and a setTimeout callback. Explain why a busy loop delays a timer.

6 | Error Handling in Backend Applications

6.1 Synchronous errors

```
function divide(a, b) {
  if (b === 0) throw new Error('Cannot divide by zero');
  return a / b;
}
try {
  console.log(divide(10, 0));
} catch (error) {
  console.log('Error:', error.message);
} finally {
  console.log('Finished');
}
```

A thrown error interrupts normal execution until caught. Error objects commonly provide `name`, `message` and `stack`. Do not expose internal stacks to API clients.

6.2 Rejected Promises

```
async function getUser(id) {
  if (id !== 1) throw new Error('User not found');
  return { id: 1, name: 'Rahim' };
}
async function main() {
  try {
    await getUser(999);
  } catch (error) {
    console.log(error.message);
  }
}
main();
```

A synchronous try/catch does not catch an unawaited rejected Promise. Attach .catch(), await the Promise inside a try block, or return it to an environment with defined rejection handling. Similarly, an outer try/catch does not catch exceptions thrown later from a detached timer callback.

6.3 Custom AppError

```
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.name = 'AppError';
    this.statusCode = statusCode;
    this.isOperational = true;
  }
}
throw new AppError('User not found', 404);
```

`super(message)` initializes the parent Error. Custom fields such as `statusCode` are application conventions; Node does not interpret them automatically.

6.4 HTTP status code reference

Code	Meaning	Typical use
200	OK	Successful read/update
201	Created	Created a resource
400	Bad Request	Malformed/invalid request
401	Unauthorized	Missing/invalid authentication
403	Forbidden	Insufficient permissions
404	Not Found	Missing resource
409	Conflict	Duplicate unique field
422	Unprocessable Content	Semantically invalid data
429	Too Many Requests	Rate limit
500	Internal Server Error	Unexpected server failure
503	Service Unavailable	Temporary unavailability

6.5 Express 5 centralized error handling

```
import express from 'express';
const app = express();
app.use(express.json());

app.get('/users/:id', async (req, res) => {
  const id = Number(req.params.id);
  if (!Number.isSafeInteger(id) || id <= 0) {
    throw new AppError('Invalid user ID', 400);
  }
  const user = id === 1 ? { id: 1, name: 'Rahim' } : null;
  if (!user) throw new AppError('User not found', 404);
  res.json({ success: true, data: user });
});

app.use((req, res, next) => {
  next(new AppError('Route not found', 404));
});
app.use((err, req, res, next) => {
  if (res.headersSent) return next(err);
  const expected = err instanceof AppError;
  if (!expected) console.error(err);
  res.status(expected ? err.statusCode : 500).json({
    success: false,
    message: expected ? err.message : 'Internal server error'
  });
});
app.listen(3000);
```

Express 5 forwards rejected Promises returned by async route handlers to error middleware. The error middleware has four parameters and belongs after route definitions. Detached timer callbacks still need explicit error handling.

6.6 Operational failures vs programming defects

Expected / operational	Programming defect
User not found	Accessing a property of undefined
Invalid request data	Wrong variable or assumption
Expired token	Invalid internal invariants
Temporary dependency outage	Faulty implementation logic

Log unexpected errors securely, validate user inputs, use database uniqueness constraints, avoid leaking secrets and avoid indiscriminate retries. Critical process-level failures may require a controlled shutdown and supervised restart.

Exercise: Add POST /users with 400 for invalid input, 409 for duplicate email and 201 for success. Make all errors use a common JSON shape.

7 | Modern Syntax and Configuration

7.1 Object destructuring

```
const user = { id: 1, name: 'Rahim', role: 'admin' };
const { name, role } = user;
const { name: username } = user; // rename variable
const { city = 'Unknown' } = user; // default if undefined
console.log(name, role, username, city);
// Rahim admin Rahim Unknown
```

7.2 Arrays and nested values

```
const numbers = [10, 20, 30];
const [first, , third] = numbers;
const addressUser = { address: { city: 'Dhaka' } };
const { address: { city } } = addressUser;
console.log(first, third, city); // 10 30 Dhaka
```

A nested destructuring expression can fail when an intermediate property is undefined. Use optional chaining or validate structure when input is uncertain.

7.3 Spread and rest syntax

```
const values = [1, 2, 3];
const copied = [...values];
const user = { id: 1, name: 'Rahim', role: 'user' };
const updated = { ...user, role: 'admin' };
const { id, ...rest } = user;
function sum(...nums) {
  return nums.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3)); // 6
```

Spread expands and copies properties/elements; rest collects remaining values. Object spread performs a shallow copy. Do not spread raw client updates into sensitive records without an explicit allowlist.

7.4 Optional chaining and nullish coalescing

```
const user = { name: 'Rahim' };
console.log(user.profile?.city); // undefined
console.log(user.profile?.city ?? 'Unknown');
// Unknown
console.log(0 || 10);  // 10
console.log(0 ?? 10);  // 0
console.log(false ?? true); // false
```

`?.` stops access when the left-hand value is null/undefined. `??` supplies a fallback only for null/undefined, unlike `||`, which treats all falsy values as absent. Neither operator replaces request validation.

7.5 Environment variables

```
# .env  (never commit production secrets)
PORT=3000
NODE_ENV=development
DATABASE_URL=postgresql://localhost:5432/taskflow

# run from terminal
node --env-file=.env server.js
```

```
// src/config/env.js
const PORT = Number(process.env.PORT ?? 3000);
if (!Number.isSafeInteger(PORT) || PORT < 0 || PORT > 65535) {
  throw new Error('Invalid PORT configuration');
}
export const config = {
  port: PORT,
  nodeEnv: process.env.NODE_ENV ?? 'development'
};
```

Environment variables are strings. Parse and validate typed values. Keep `.env` files out of Git, document required keys in `.env.example`, and use a secure secrets manager for sensitive deployment credentials.

```
# .gitignore
node_modules/
.env
.env.*
!.env.example
```

7.6 Modular Express project layout

```
taskflow-api/
  src/
    config/env.js
    controllers/task.controller.js
    routes/task.routes.js
    app.js
  server.js
  package.json
  .env
  .env.example
  .gitignore
```

Route: matches HTTP method and path. Controller: handles request data, validation and response. App: configures middleware and mounts routes. Server: listens on a port. In the PostgreSQL phase, data access will move into a separate database/service layer.

7.7 Working minimal TaskFlow API

```
// src/controllers/task.controller.js
const tasks = [
  { id: 1, title: 'Learn Node.js', completed: false }
];
export function getAllTasks(req, res) {
  res.json({ success: true, data: tasks });
}
export function getTaskById(req, res) {
  const id = Number(req.params.id);
  if (!Number.isSafeInteger(id) || id <= 0) {
    return res.status(400).json({ message: 'Invalid task ID' });
  }
  const task = tasks.find(t => t.id === id);
  if (!task) return res.status(404).json({ message: 'Not found' });
  res.json({ success: true, data: task });
}
export function createTask(req, res) {
  const { title, completed = false } = req.body ?? {};
  if (typeof title !== 'string' || !title.trim() ||
      typeof completed !== 'boolean') {
    return res.status(400).json({ message: 'Invalid task' });
  }
  const task = {
    id: Math.max(0, ...tasks.map(t => t.id)) + 1,
    title: title.trim(), completed
  };
  tasks.push(task);
  res.status(201).json({ success: true, data: task });
}
```

```
// src/routes/task.routes.js
import { Router } from 'express';
import {
  getAllTasks, getTaskById, createTask
} from '../controllers/task.controller.js';
const router = Router();
router.get('/', getAllTasks);
router.get('/:id', getTaskById);
router.post('/', createTask);
export default router;
```

```
// src/app.js
import express from 'express';
import taskRoutes from './routes/task.routes.js';
const app = express();
app.use(express.json());
app.use('/tasks', taskRoutes);
app.use((req, res) => res.status(404).json({
  success: false, message: 'Route not found'
}));
app.use((err, req, res, next) => {
  if (res.headersSent) return next(err);
  console.error(err);
  res.status(500).json({
    success: false, message: 'Internal server error'
  });
});
```

```
export default app;

// server.js
import app from './src/app.js';
import { config } from './src/config/env.js';
app.listen(config.port, () => {
  console.log(`Server running on port ${config.port}`);
});
```

```
# Setup
npm init -y
npm install express@5
npm pkg set type=module
node --env-file=.env server.js

# Check the endpoints
curl http://localhost:3000/tasks
curl http://localhost:3000/tasks/1
curl -X POST http://localhost:3000/tasks \
  -H 'Content-Type: application/json' \
  -d '{"title":"Learn PostgreSQL"}'
```

In-memory tasks disappear at restart. The incremental max-ID strategy is for a simple exercise, not production concurrency. Later we will use PostgreSQL-generated identifiers, uniqueness constraints and transactions.

Appendix A | Output Prediction Practice

Questions - answer before reading the key

```
// Q1. Promise executor vs handler
console.log('A');
const p = new Promise(resolve => {
  console.log('B');
  resolve('C');
});
p.then(v => console.log(v));
console.log('D');

// Q2. Async continuation
async function test() {
  console.log('A');
  await Promise.resolve();
  console.log('B');
}
console.log('C');
test();
console.log('D');

// Q3. Nullish values
console.log(0 || 5);
console.log(0 ?? 5);
console.log('' || 'default');
console.log('' ?? 'default');
```

Answers and reasoning

- Q1: A, B, D, C. The Promise executor runs synchronously; .then is a microtask.
- Q2: C, A, D, B. The function starts immediately; await suspends its continuation.
- Q3: 5, 0, "default", "". Nullish coalescing preserves falsy non-nullish values.

Bonus questions

```
// Q4. Shallow copy
const a = { profile: { age: 20 } };
const b = { ...a };
b.profile.age = 30;
console.log(a.profile.age);

// Q5. Timer and microtask
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
```

- Q4: 30; nested objects retain shared references.
- Q5: 1, 4, 3, 2; synchronous work, then Promise microtask, then timer.

Appendix B | Backend Quick Reference

Pattern	Use
export / import	Share functionality across ES Module files
fn(callback)	Supply work for another function to invoke
new Promise(resolve, reject)	Represent an eventual outcome
.then().catch()	Consume success/failure without await
async function	Function always returns a Promise
await promise	Suspend current async function until settlement
Promise.all([...])	Run independent asynchronous operations concurrently
try/catch/finally	Handle exceptions and perform cleanup
throw new Error(...)	Signal a failure
next(error)	Forward an error into Express error handling
const {a,b}=obj	Object destructuring
const [a,b]=arr	Array destructuring
{...obj, key:value}	Shallow copy / object update
function f(...args)	Collect rest arguments
user?.profile	Safe nullish property access
value ?? fallback	Default only for null/undefined
process.env.PORT	Read runtime configuration

Common errors and fixes

Problem	Fix
Unexpected Promise printed	Add await or .then when a resolved value is needed.
Unhandled rejection	Await in try/catch or attach a rejection handler.
Cannot use import statement	Set "type": "module" or use .mjs.
Cannot find module	Check file path and .js extension in ESM.
Timer executes late	Check for CPU blocking and microtask starvation.
undefined property access	Validate input or use ?. appropriately.
Default overwrites 0 or false	Use ?? instead of || where appropriate.
Duplicate email race	Use a PostgreSQL UNIQUE constraint.
Raw error sent to client	Return safe message; log internally.
.env committed to Git	Ignore it and rotate any exposed secrets.

Appendix C | Final Practice Project

TaskFlow PATCH /tasks/:id - requirements

- Accept optional title and completed fields, but at least one permitted field must be supplied.
- Reject invalid IDs with 400; reject missing tasks with 404.
- Reject unknown fields such as role, ownerId or isAdmin.
- Validate title as a non-empty string and completed as a boolean.
- Leave omitted fields unchanged; return 200 and the modified task.
- Test with valid update, invalid ID, missing task, invalid type and forbidden field.

Sample implementation (add to controller)

```
export function updateTask(req, res) {
  const id = Number(req.params.id);
  if (!Number.isSafeInteger(id) || id <= 0) {
    return res.status(400).json({ message: 'Invalid task ID' });
  }
  const task = tasks.find(t => t.id === id);
  if (!task) return res.status(404).json({ message: 'Not found' });
  const body = req.body;
  if (!body || typeof body !== 'object' || Array.isArray(body)) {
    return res.status(400).json({ message: 'Invalid JSON body' });
  }
  const keys = Object.keys(body);
  if (keys.length === 0 ||
      keys.some(k => !['title', 'completed'].includes(k))) {
    return res.status(400).json({ message: 'Invalid fields' });
  }
  const { title, completed } = body;
  if ((title !== undefined &&
      (typeof title !== 'string' || !title.trim())) ||
      (completed !== undefined && typeof completed !== 'boolean')) {
    return res.status(400).json({ message: 'Invalid values' });
  }
  if (title !== undefined) task.title = title.trim();
  if (completed !== undefined) task.completed = completed;
  res.json({ success: true, data: task });
}

// Add to routes: import updateTask, then:
router.patch('/:id', updateTask);
```

Extra practice tasks

- Add GET /health that returns {status:"ok"} without a database call.
- Add a 400 response for a malformed task ID, such as /tasks/abc.
- Try Promise.all with two simulated operations taking 1000 and 2000 ms.
- Extract the PORT configuration into a separate module and validate it.
- Write three test cases for 201, 400 and 404 API responses.

Readiness checklist for Phase 2

- I can explain callback vs Promise vs async/await.
- I can predict ordinary synchronous/microtask/timer output order.
- I can explain why asynchronous I/O differs from parallel CPU execution.
- I can handle expected HTTP failures without leaking internal errors.
- I can split a small Express app into modules, routes and controllers.
- I can use destructuring, spread/rest, optional chaining and nullish defaults.
- I can configure a project using environment variables without committing secrets.

Next phase: Node.js internals, built-in modules, npm/package.json, filesystem APIs, events, buffers, streams and native HTTP servers. The TaskFlow project will continue evolving as we learn Express, PostgreSQL and other backend technologies.